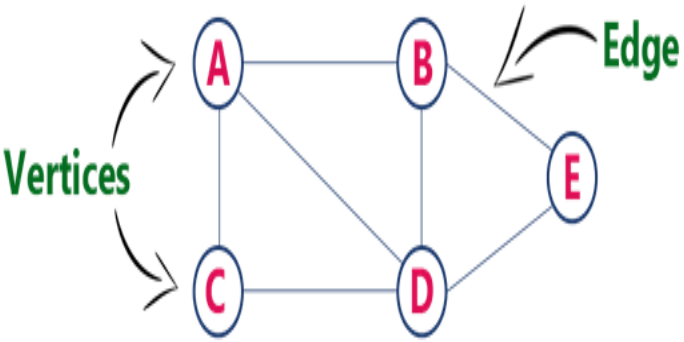
UNIT 4 – GRAPH DATA STRUCTURES

Graph Terminology – Directed Graphs – Representation of Graphs – Graph Traversal Algorithms – Breadth First Search Algorithm – Depth First Search Algorithm – Shortest Path Algorithms: Minimum spanning tree – Prim's Algorithm – Kruskal's Algorithm- Dijkstra's Algorithm – Applications of Graphs.

DEFINITION OF GRAPH

- A graph is defined as Graph is a collection of vertices and arcs which connects vertices in the graph.
- A graph G is represented as G = (V , E), where V is set of vertices and E is set of edges.
- Example: graph G can be defined as G = (V , E) Where V = {A,B,C,D,E} and E = {(A,B),(A,C)(A,D),(B,D),(C,D),(B,E),(E,D)}.

This is a graph with 5 vertices and 6 edges.



GRAPH TERMINOLOGY

1. **Vertex :** An individual data element of a graph is called as Vertex. Vertex is also known as node. In above example graph, A, B, C, D & E are known as vertices.
2. **Edge :** An edge is a connecting link between two vertices. Edge is also known as Arc. An edge is represented as (starting Vertex, ending Vertex).

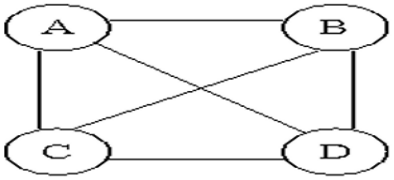
In above graph, the link between vertices A and B is represented as (A,B).

Edges are three types:

1. **Undirected Edge** - An undirected edge is a bidirectional edge. If there is an undirected edge between vertices A and B then edge (A , B) is equal to edge (B , A).
2. **Directed Edge** - A directed edge is a unidirectional edge. If there is a directed edge between vertices A and B then edge (A , B) is not equal to edge (B , A).
3. **Weighted Edge** - A weighted edge is an edge with cost on it.

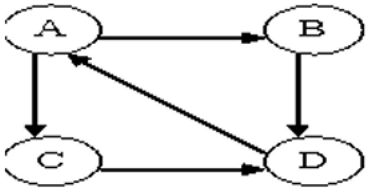
Types of Graphs

1.Undirected Graph :A graph with only undirected edges is said to be undirected graph.



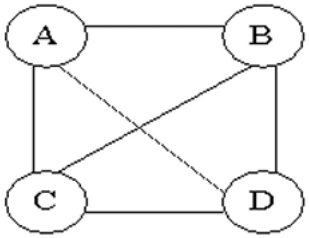
Undirected Graph.

2.Directed Graph: A graph with only directed edges is said to be directed graph.



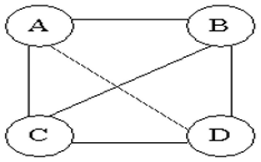
Directed Graph.

3.Complete Graph: A graph in which any V node is adjacent to all other nodes present in the graph is known as a complete graph. An undirected graph contains the edges that are equal to edges = n(n-1)/2 where n is the number of vertices present in the graph.



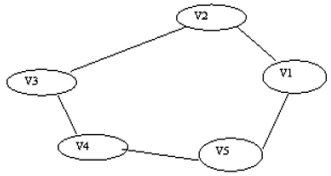
A complete graph.

4.Regular Graph : Regular graph is the graph in which nodes are adiacent to each other, i.e., each node is accessible from any other



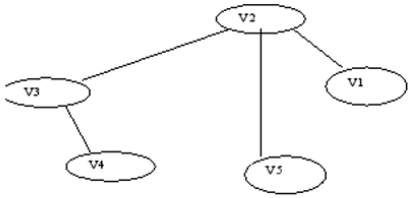
A regular graph

5.Cycle Graph :A graph having cycle is called cycle graph. In this case the first and last nodes are the same. A closed simple path is a cycle



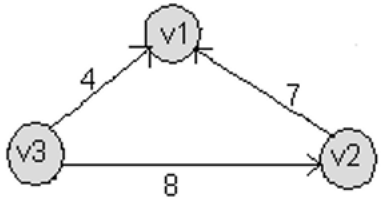
A cycle graph

6.Acyclic Graph: A graph without cycle is called acyclic graph



A acyclic graph

7. Weighted Graph :A graph is said to be weighted if there are some non negative value assigned to each edges of the graph. The value is equal to the length between two vertices. Weighted graph is also called a network.



A weighted graph

Outgoing Edge: A directed edge is said to be outgoing edge on its orign vertex.

Incoming Edge: A directed edge is said to be incoming edge on its destination vertex.

Degree : Total number of edges connected to a vertex is said to be degree of that vertex.

Indegree: Total number of incoming edges connected to a vertex is said to be indegree of that vertex.

Outdegree: Total number of outgoing edges connected to a vertex is said to be outdegree of that vertex.

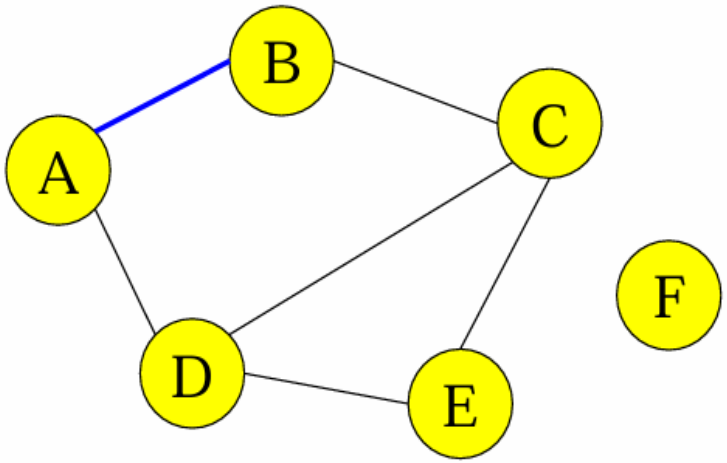
Parallel edges or Multiple edges: If there are two undirected edges to have the same end vertices, and for two directed edges to have the same origin and the same destination. Such edges are called parallel edges or multiple edges.

Self-loop: An edge (undirected or directed) is a self-loop if its two endpoints coincide.

Graph Representations

- Space and time are analyzed in terms of:
  - Number of vertices = $|V|$ and
  - Number of edges = $|E|$
- There are at least two ways of representing graphs:
  - The *adjacency matrix* representation
  - The *adjacency list* representation

Adjacency Matrix

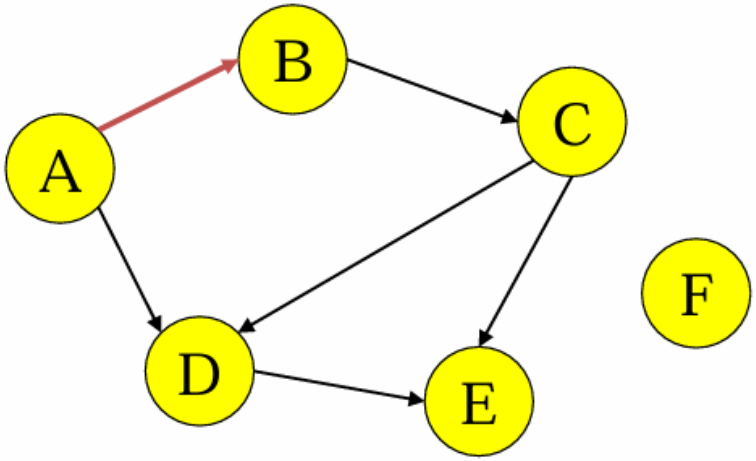


$$M(v, w) = \begin{cases} 1 \text{ if } (v, w) \text{ is in E} \\ 0 \text{ otherwise} \end{cases}$$

	A	B	C	D	E	F
A	0	1	0	1	0	0
B	1	0	1	0	0	0
C	0	1	0	1	1	0
D	1	0	1	0	1	0
E	0	0	1	1	0	0
F	0	0	0	0	0	0

Space = $|V|^2$

Adjacency Matrix for a Digraph



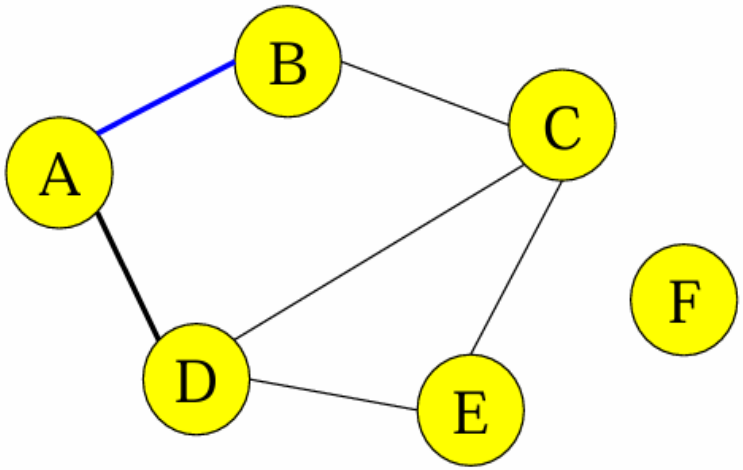
$$M(v, w) = \begin{cases} 1 \text{ if } (v, w) \text{ is in E} \\ 0 \text{ otherwise} \end{cases}$$

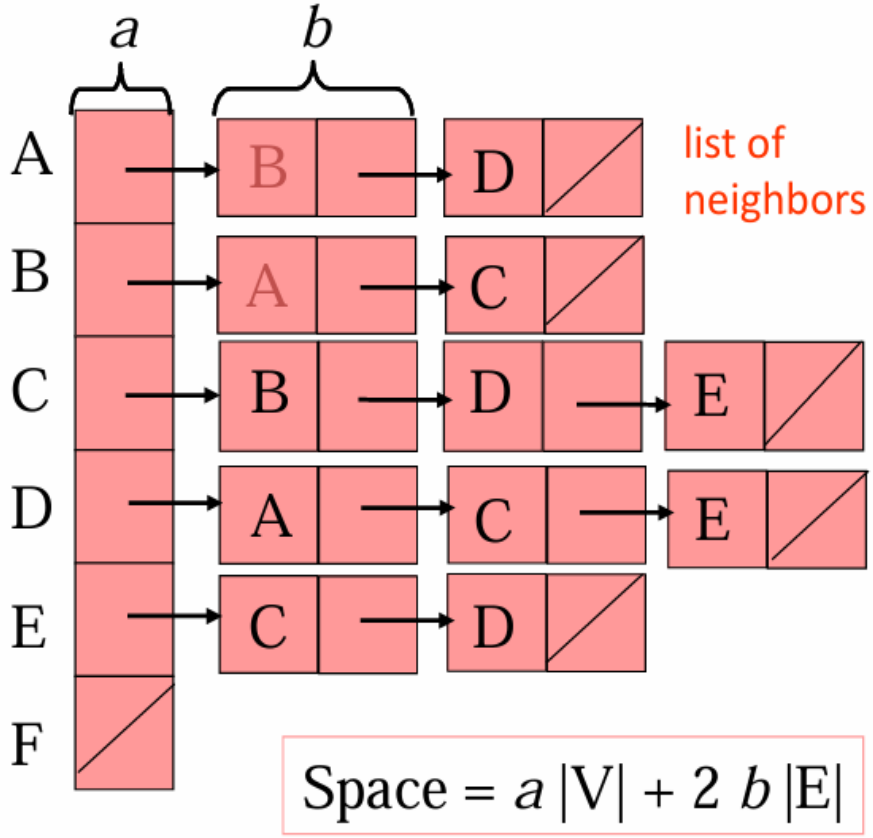
	A	B	C	D	E	F
A	0	1	0	1	0	0
B	0	0	1	0	0	0
C	0	0	0	1	1	0
D	0	0	0	0	1	0
E	0	0	0	0	0	0
F	0	0	0	0	0	0

Space = $|V|^2$

Adjacency List

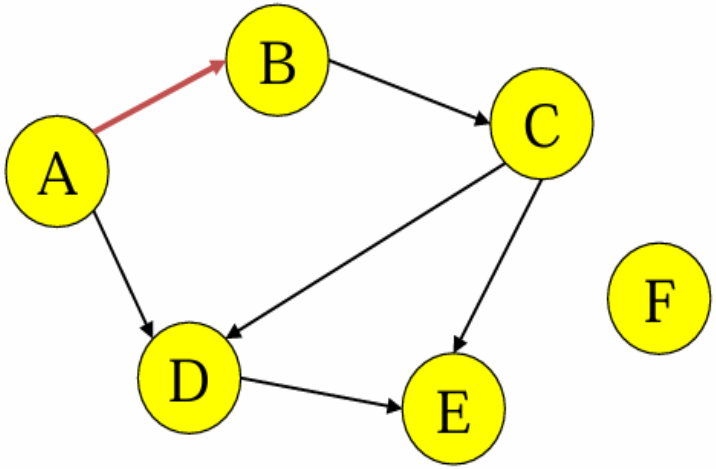
For each *v* in *V*, *L*(*v*) = list of *w* such that (*v*, *w*) is in *E*

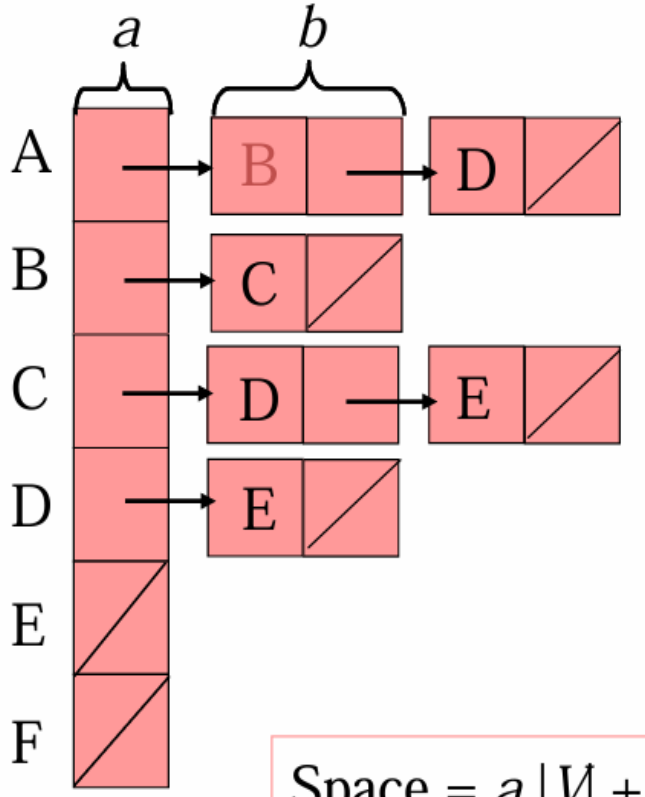




Adjacency List for a Digraph

For each *v* in *V*, *L*(*v*) = list of *w* such that (*v*, *w*) is in *E*





Space = $a|V| + b|E|$

Graph Traversals

- Graph traversal is a technique used for searching a vertex in a graph.
- The graph traversal is also used to decide the order of vertices is visited in the search process.
- A graph traversal finds the edges to be used in the search process without creating loops.
- There are two graph traversal techniques and they are as follows...
- **DFS (Depth First Search)**
- **BFS (Breadth First Search)**

Basic Algorithm for Traversals:

- Select a starting node
- Make a set of nodes adjacent to current node
- Visit each node in the set but "mark" each nodes after visiting them so you don't revisit them (and eventually stop)
- Repeat above but skip "marked nodes"

DFS (Depth First Search)

- DFS traversal of a graph produces a **spanning tree** as final result. **Spanning Tree** is a graph without loops.
- Use **Stack data structure** with maximum size of total number of vertices in the graph to implement DFS traversal.

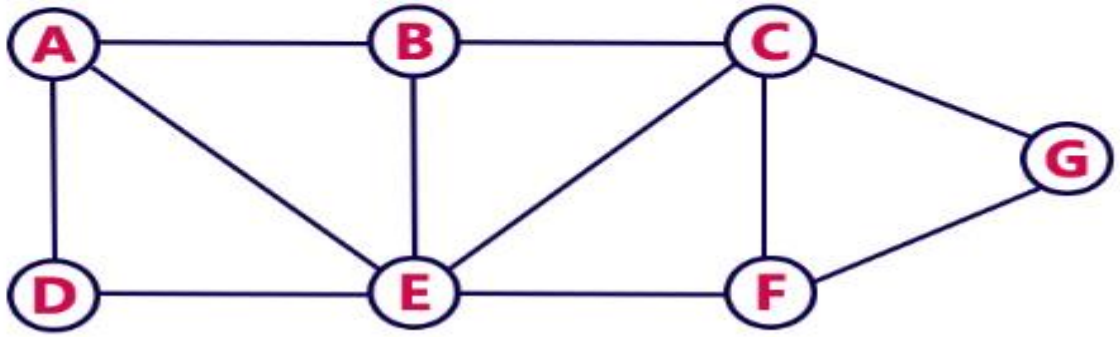
The following steps to implement DFS traversal...

- **Step 1 -** Define a Stack of size total number of vertices in the graph.
- **Step 2 -** Select any vertex as **starting point** for traversal. Visit that vertex and push it on to the Stack.
- **Step 3 -** Visit any one of the non-visited **adjacent** vertices of a vertex which is at the top of stack and push it on to the stack.
- **Step 4 -** Repeat step 3 until there is no new vertex to be visited from the vertex which is at the top of the stack.

- **Step 5 -** When there is no new vertex to visit then use **back tracking** and pop one vertex from the stack.
- **Step 6 -** Repeat steps 3, 4 and 5 until stack becomes Empty.
- **Step 7 -** When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph
- **Back tracking** is coming back to the vertex from which we reached the current vertex.

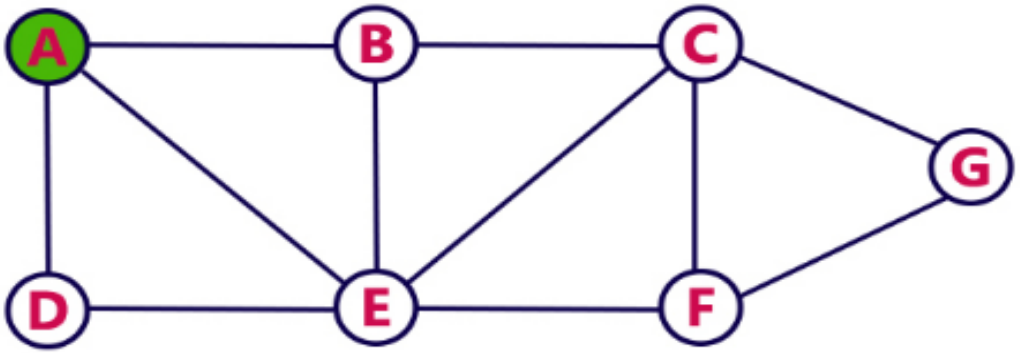
Example

Consider the following example graph to perform DFS traversal



Step 1:

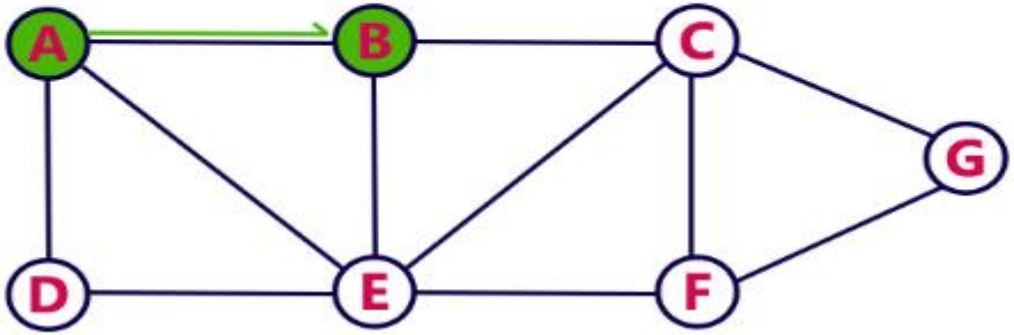
- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.

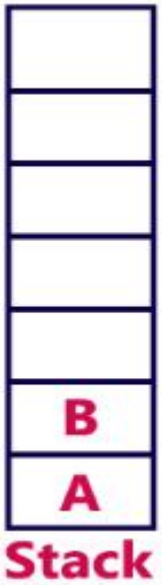




Step 2:

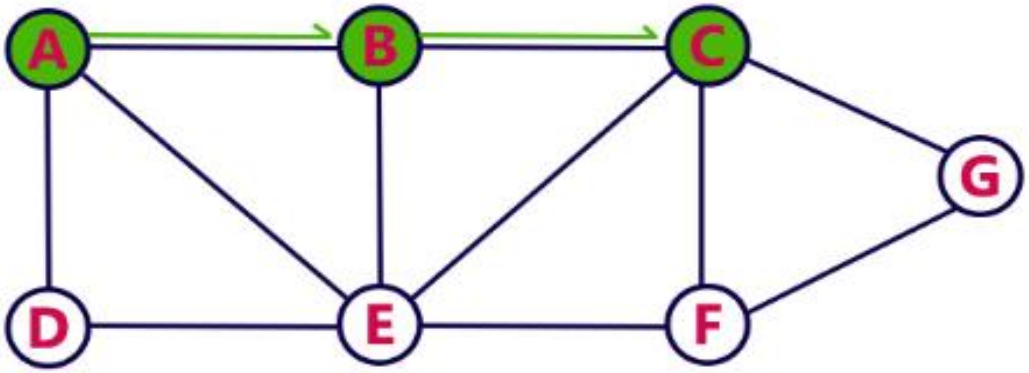
- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.

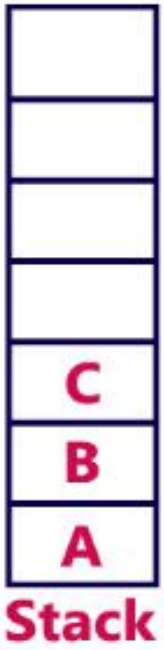




Step 3:

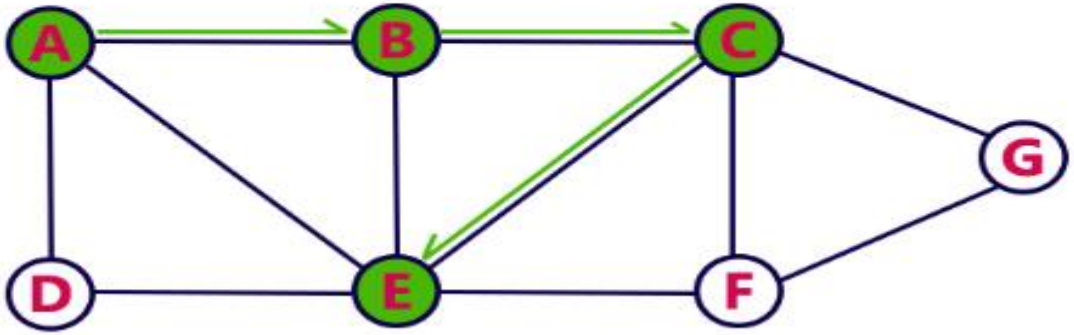
- Visit any adjacent vertext of **B** which is not visited (**C**).
- Push C on to the Stack.

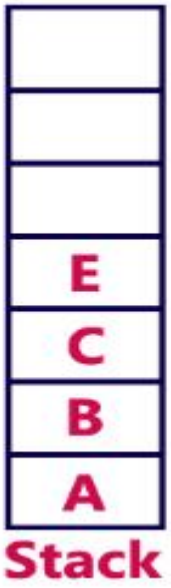




Step 4:

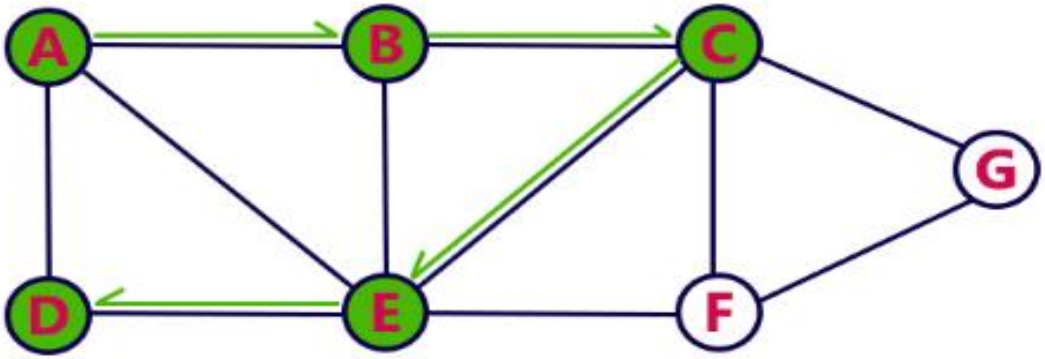
- Visit any adjacent vertext of **C** which is not visited (**E**).
- Push E on to the Stack

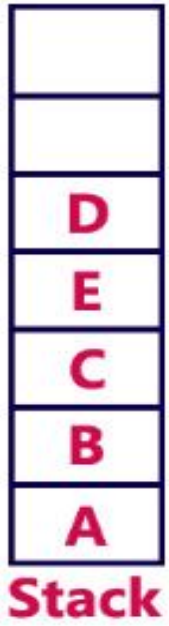




Step 5:

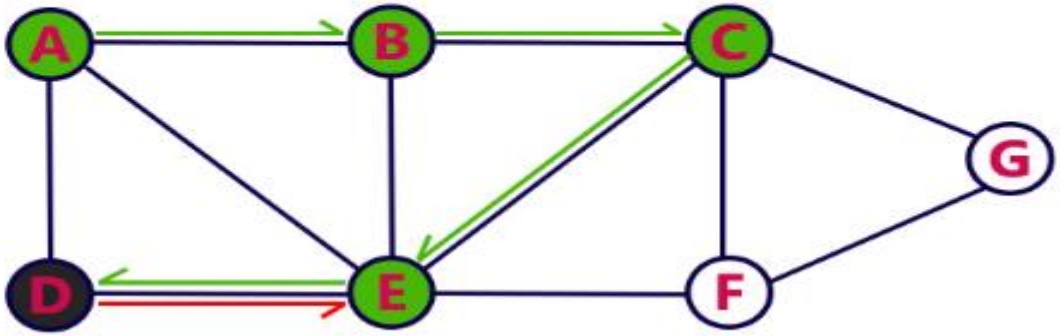
- Visit any adjacent vertext of **E** which is not visited (**D**).
- Push D on to the Stack

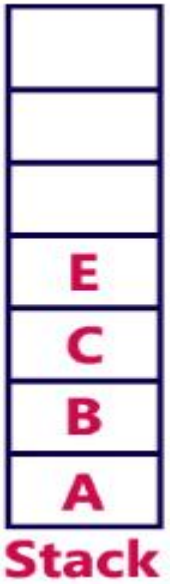




Step 6:

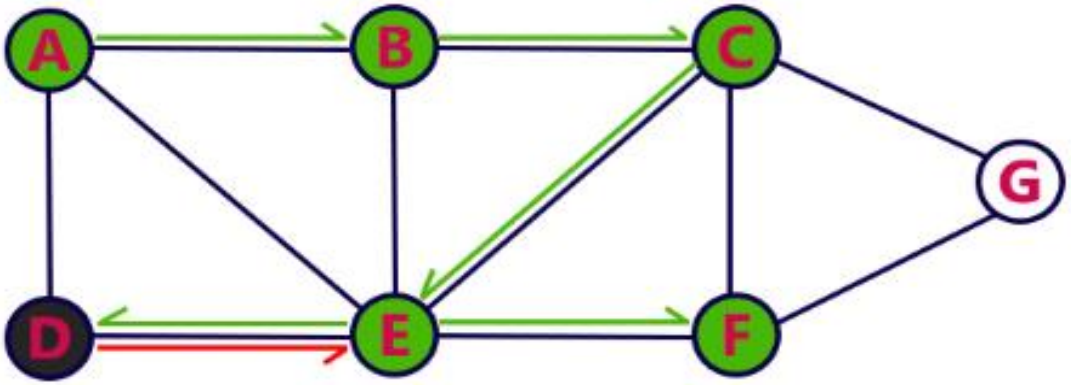
- There is no new vertiex to be visited from D. So use back track.
- Pop D from the Stack.

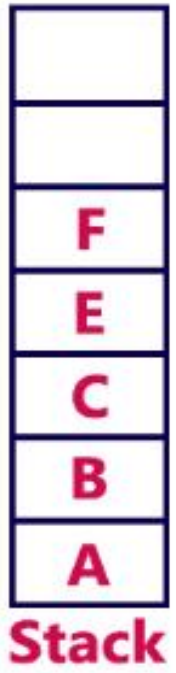




Step 7:

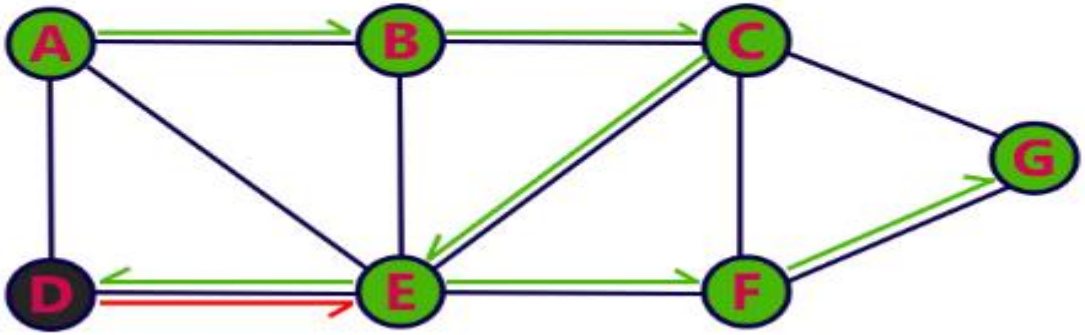
- Visit any adjacent vertex of **E** which is not visited (**F**).
- Push **F** on to the Stack.

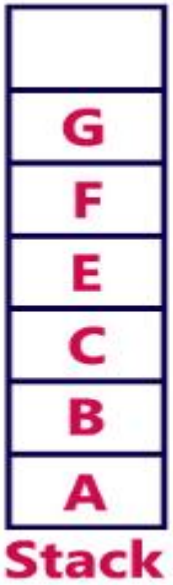




Step 8:

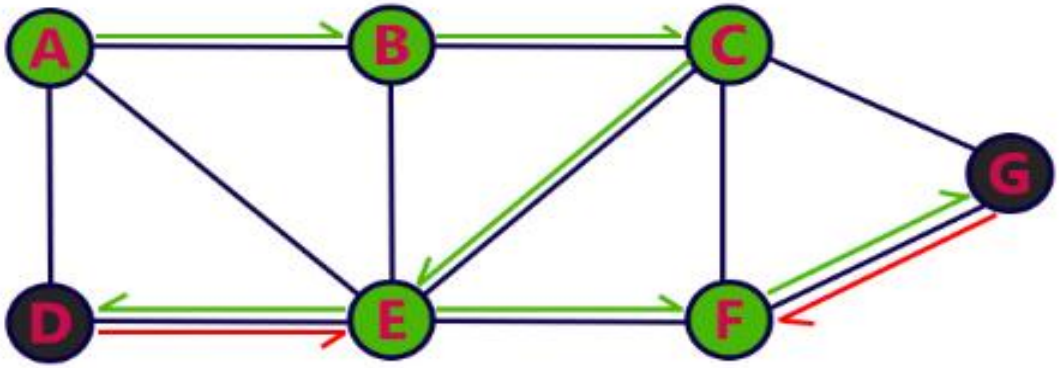
- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push **G** on to the Stack.

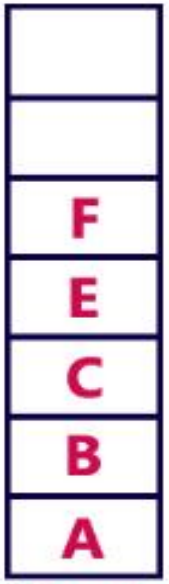




Step 9:

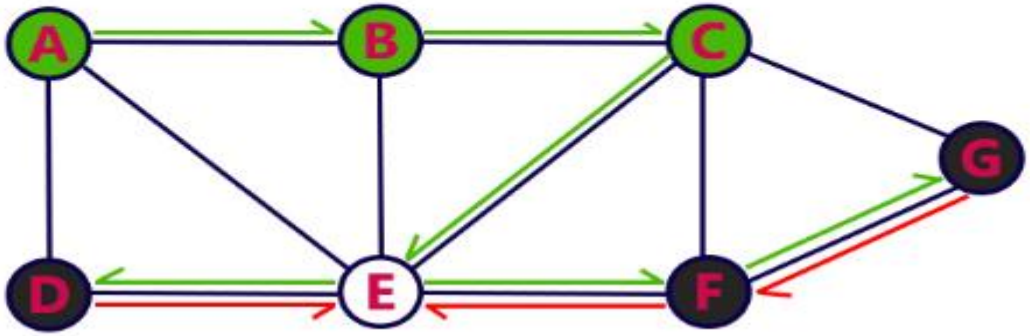
- There is no new vertiex to be visited from G. So use back track.
- Pop G from the Stack.

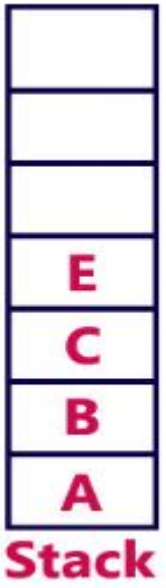




Step 10:

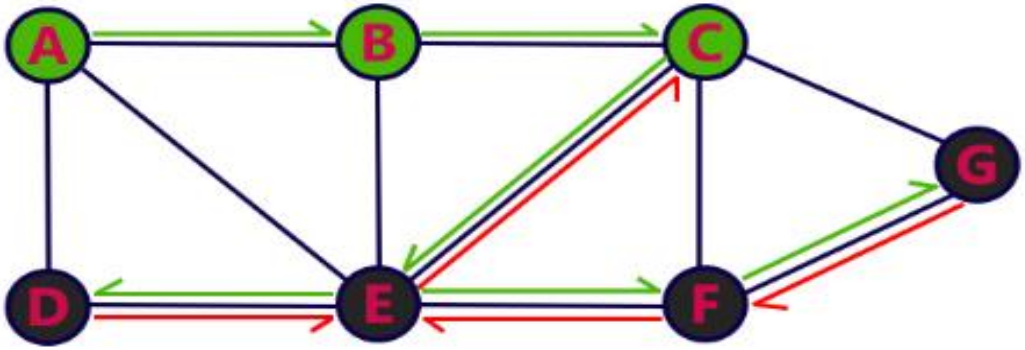
- There is no new vertiex to be visited from F. So use back track.
- Pop F from the Stack.

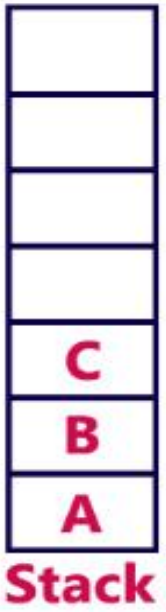




Step 11:

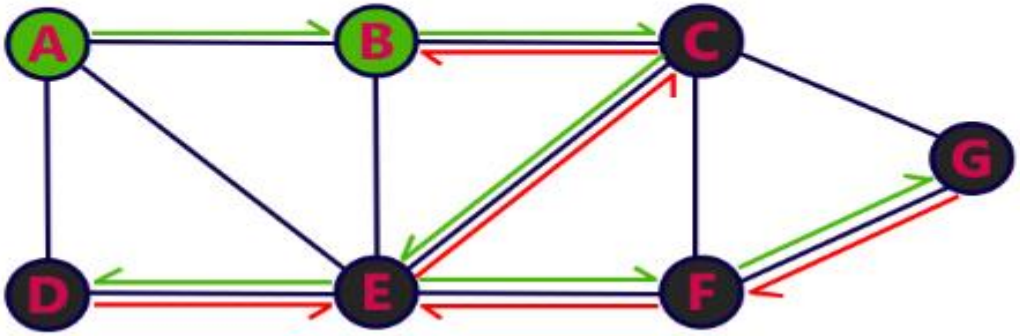
- There is no new vertiex to be visited from E. So use back track.
- Pop E from the Stack.





Step 12:

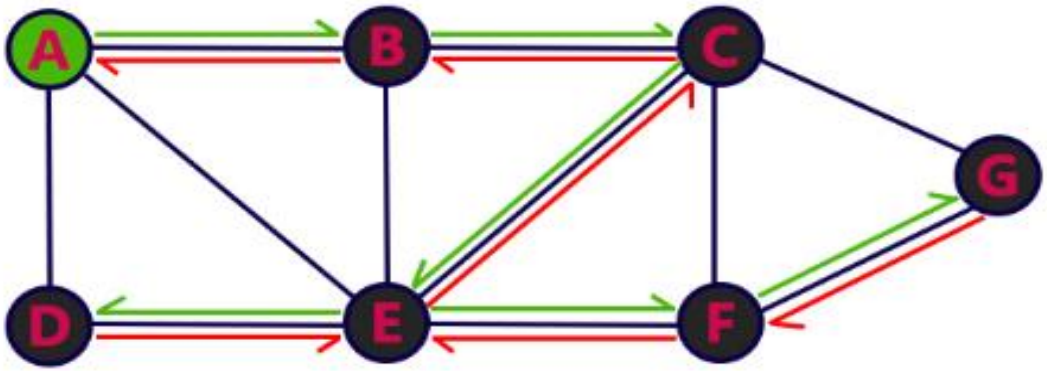
- There is no new vertiex to be visited from C. So use back track.
- Pop C from the Stack.

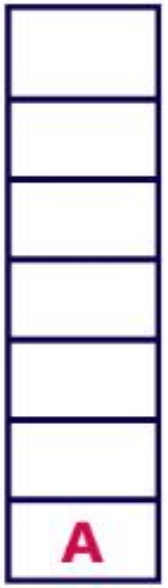




Step 13:

- There is no new vertiex to be visited from B. So use back track.
- Pop B from the Stack.

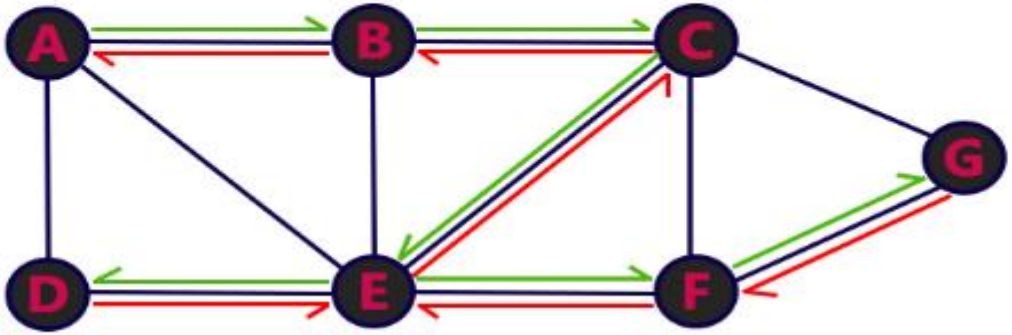


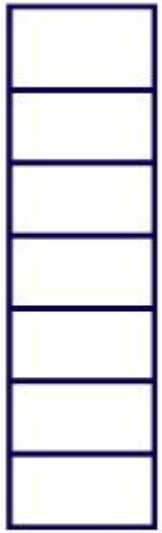


Stack

Step 14:

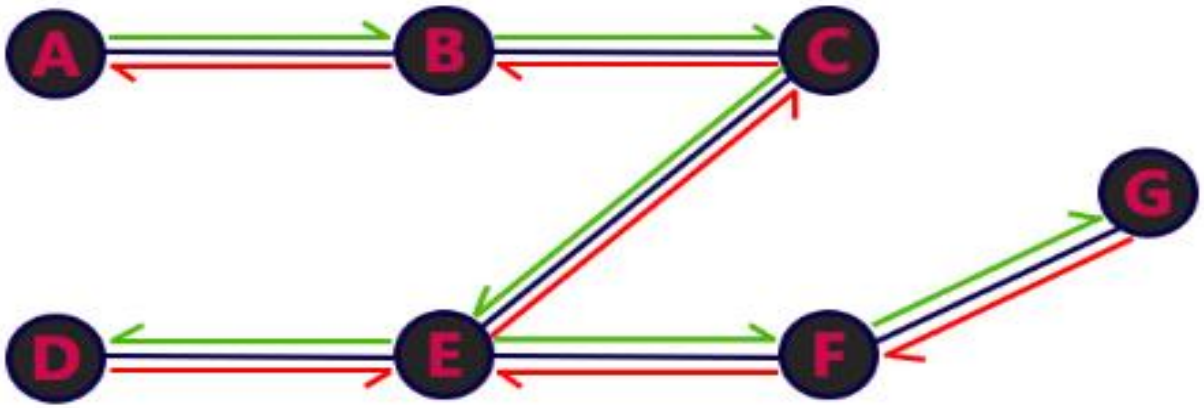
- There is no new vertiex to be visited from A. So use back track.
- Pop A from the Stack.





Stack

- Stack became Empty. So stop DFS Treversal.
- Final result of DFS traversal is following spanning tree.



BFS (Breadth First Search)

- BFS traversal of a graph produces a **spanning tree** as final result. **Spanning Tree** is a graph without loops.
- Use **Queue data structure** with maximum size of total number of vertices in the graph to implement BFS traversal.

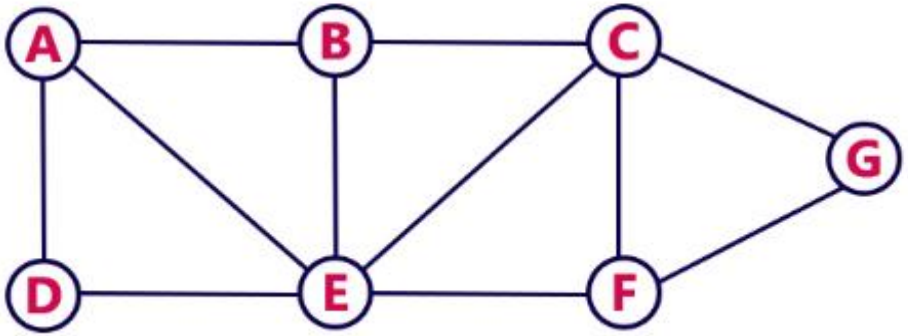
The following steps to implement BFS traversal...

- **Step 1 -** Define a Queue of size total number of vertices in the graph.
- **Step 2 -** Select any vertex as **starting point** for traversal. Visit that vertex and insert it into the Queue.
- **Step 3 -** Visit all the non-visited **adjacent** vertices of the vertex which is at front of the Queue and insert them into the Queue.

- **Step 4 -** When there is no new vertex to be visited from the vertex which is at front of the Queue then delete that vertex.
- **Step 5 -** Repeat steps 3 and 4 until queue becomes empty.
- **Step 6 -** When queue becomes empty, then produce final spanning tree by removing unused edges from the graph

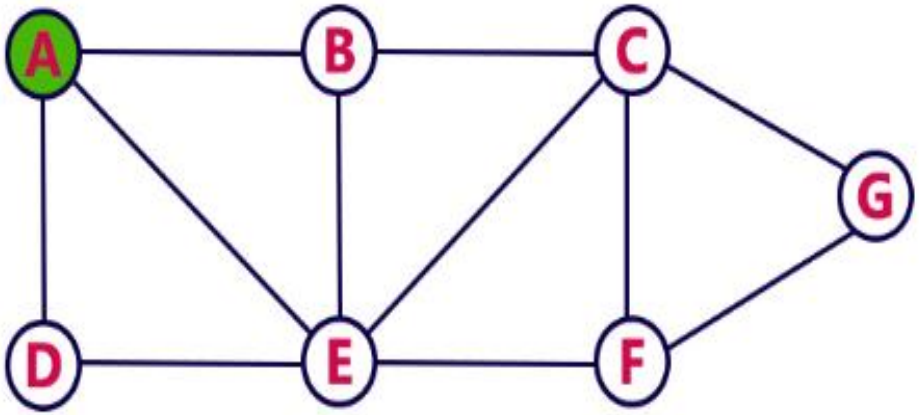
Example

Consider the following example graph to perform BFS traversal



Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.

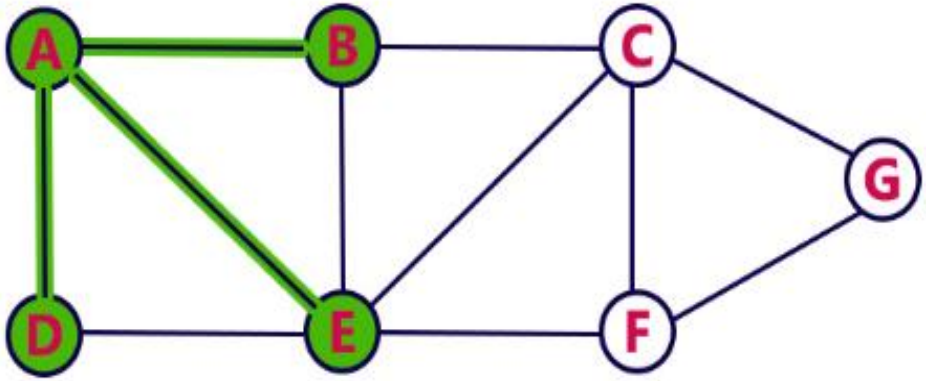


Queue

A						

Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D**, **E**, **B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..

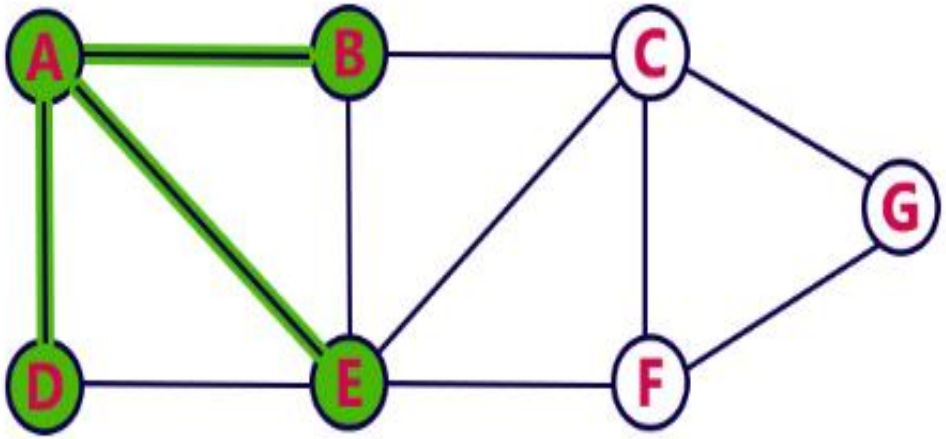


Queue

	D	E	B			

Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.

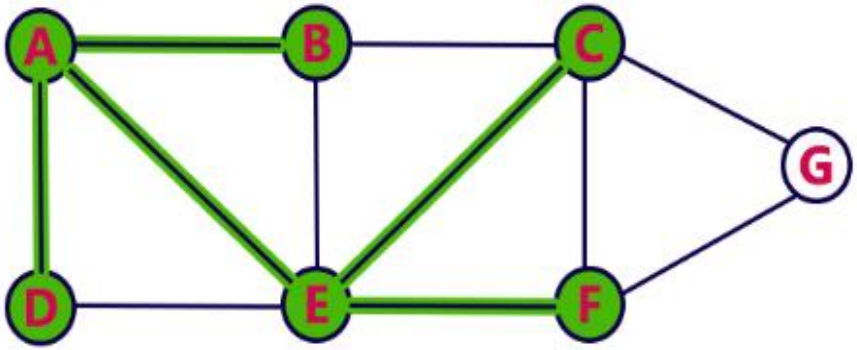


Queue

		E	B			

Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C**, **F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.

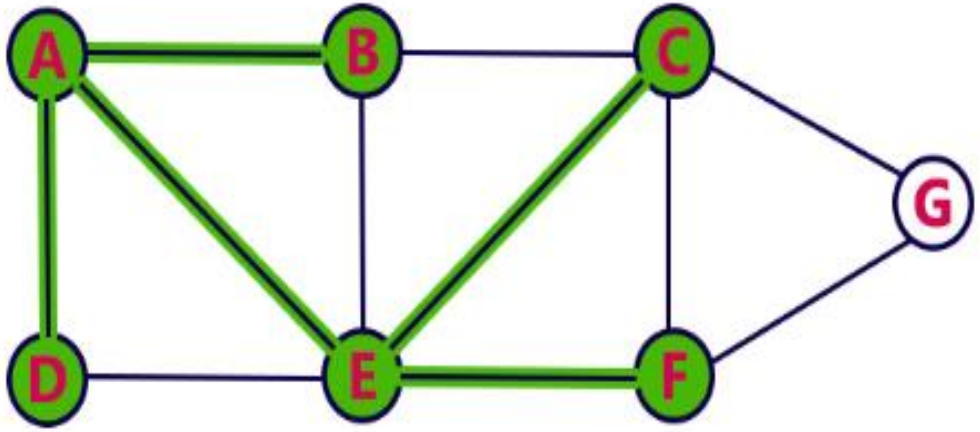


Queue

			B	C	F	

Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.

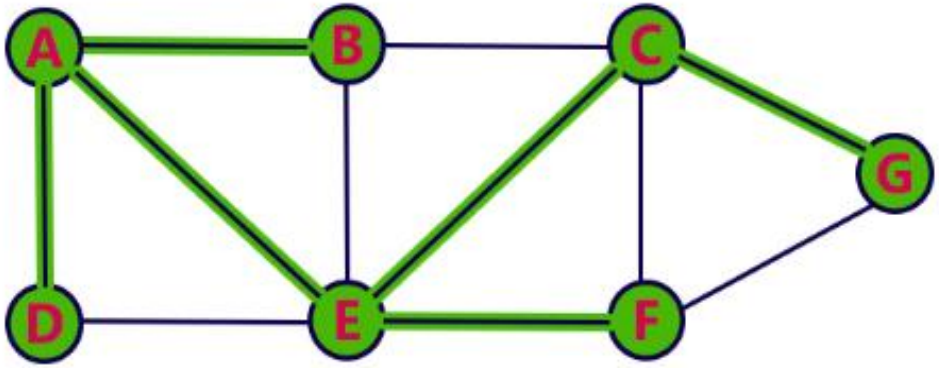


Queue

				C	F	

Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.

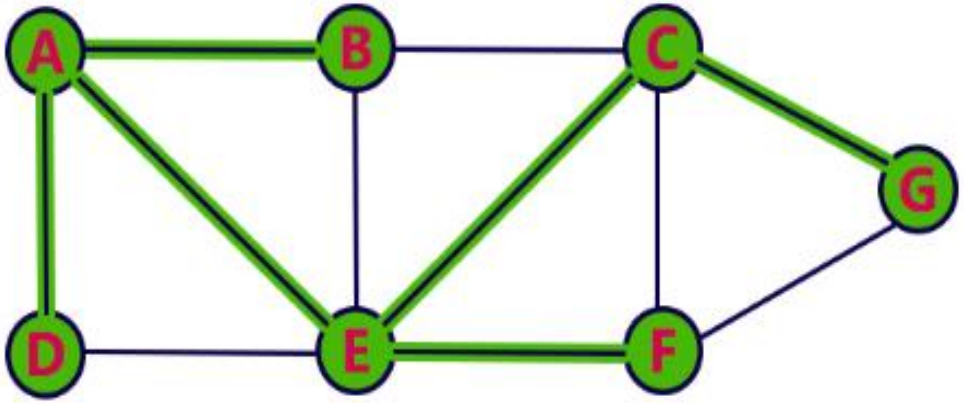


Queue

					F	G

Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.

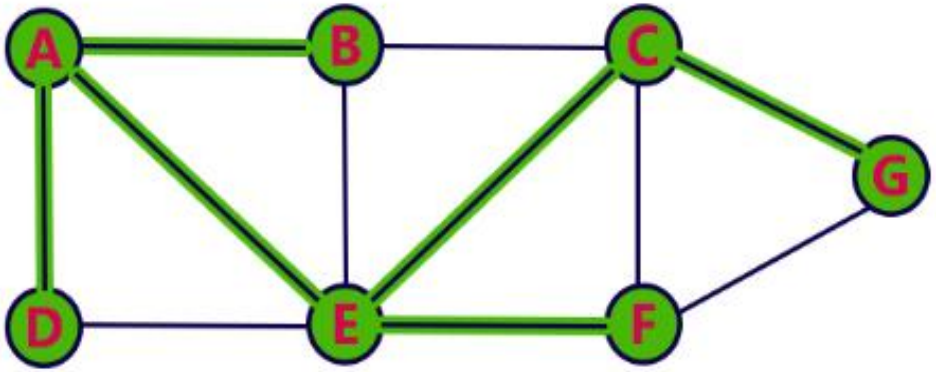


Queue

						G

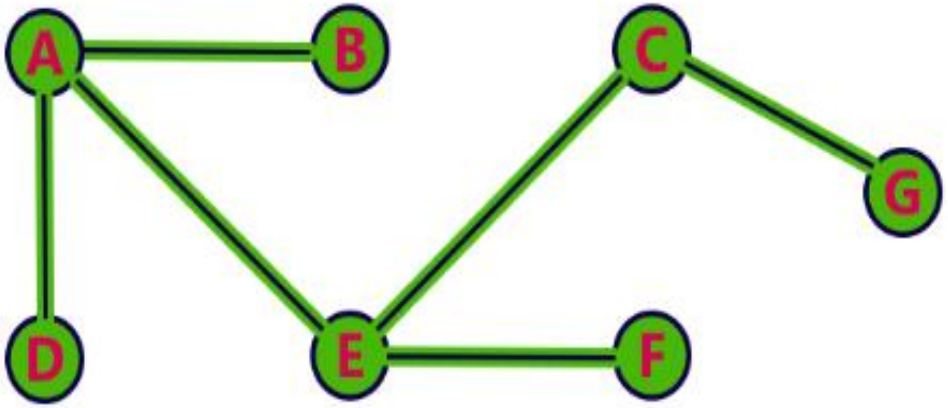
Step 8:

- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



Queue

- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



Shortest Path

The target of shortest path algorithms is to find a route between any pair of vertices along the edges, so the sum of weights of edges is minimum. If the edges are of equal weights, the shortest path algorithm aims to find a route having minimum number

Types of shortest path

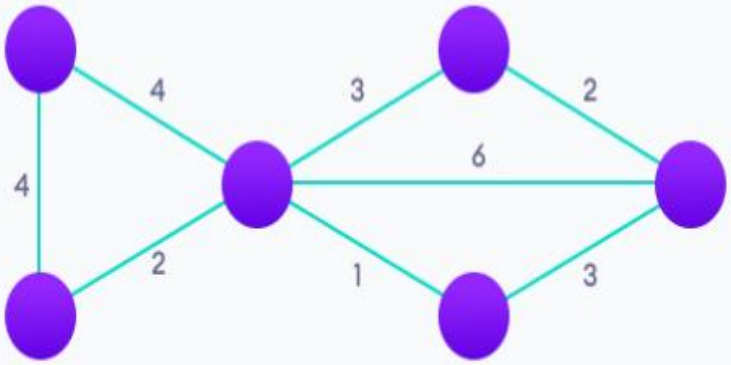
- Bellman Ford's Algorithm
- Dijkstra's Algorithm
- Floyd Warshall's Algorithm

Dijkstra's Algorithm

Dijkstra's algorithm has many variants but the most common one is to find the shortest paths from the source vertex to all other vertices in the graph.

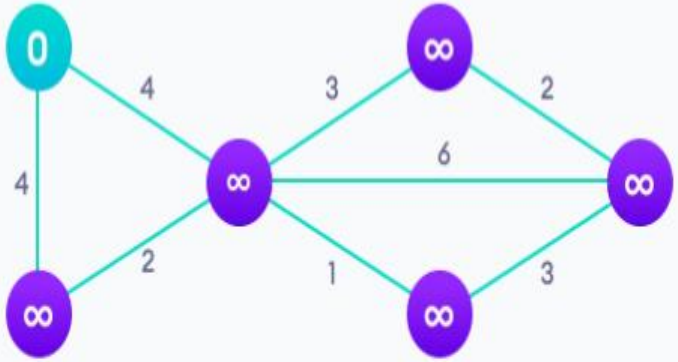
Algorithm Steps:

1. Set all vertices distances = infinity except for the source vertex, set the source distance = 0.
2. Push the source vertex in a min-priority queue in the form (distance , vertex), as the comparison in the min-priority queue will be according to vertices distances.
3. Pop the vertex with the minimum distance from the priority queue (at first the popped vertex = source).
4. Update the distances of the connected vertices to the popped vertex in case of "current vertex distance + edge weight < next vertex distance", then push the vertex
   with the new distance to the priority queue.
5. If the popped vertex is visited before, just continue without using it.
6. Apply the same algorithm again until the priority queue is empty.



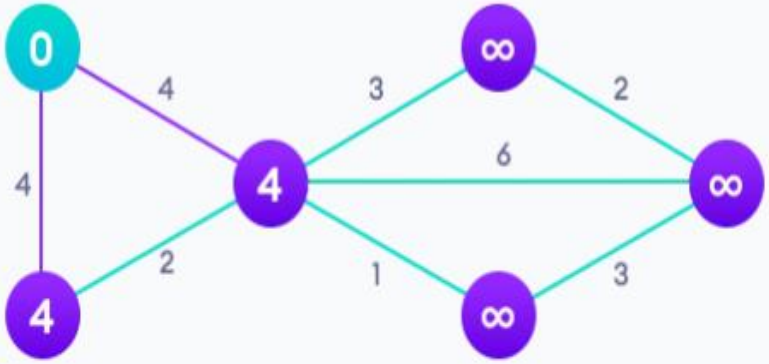
Step: 1

Start with a weighted graph



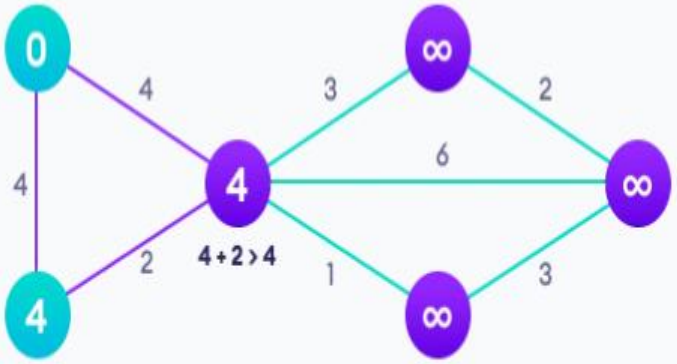
Step: 2

Choose a starting vertex and assign infinity path values to all other devices



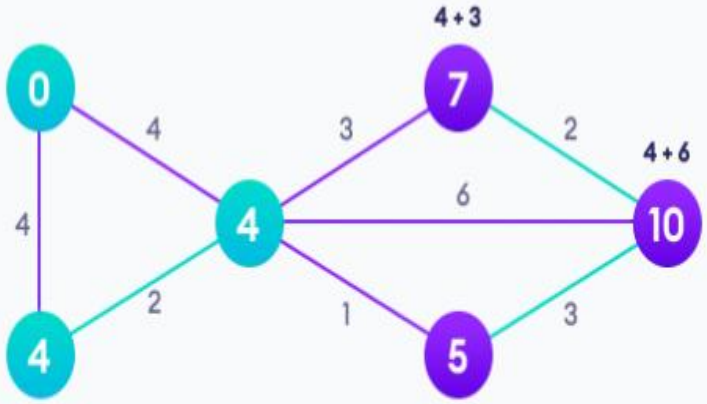
Step: 3

Go to each vertex and update its path length



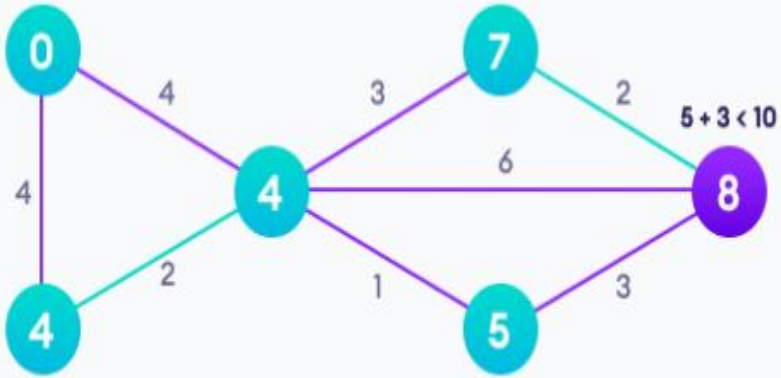
Step: 4

If the path length of the adjacent vertex is lesser than new path length, don't update it



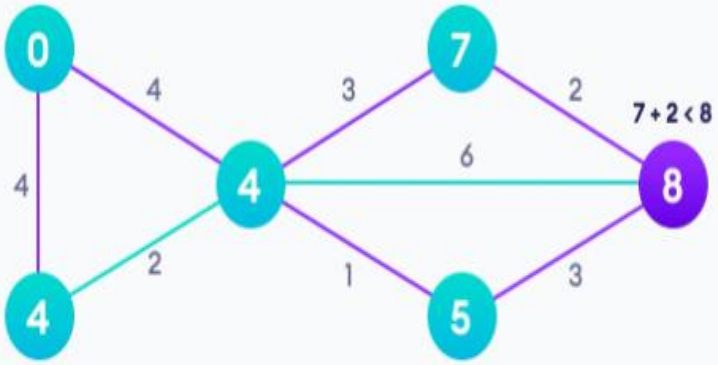
Step: 5

Avoid updating path lengths of already visited vertices



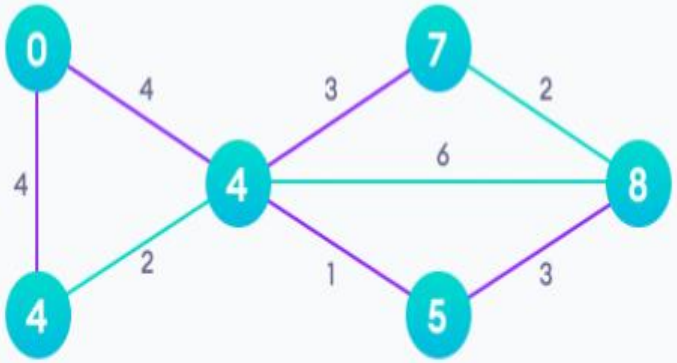
Step: 6

After each iteration, we pick the unvisited vertex with the least path length. So we choose 5 before 7



Step: 7

Notice how the rightmost vertex has its path length updated twice



Step: 8

Repeat until all the vertices have been visited

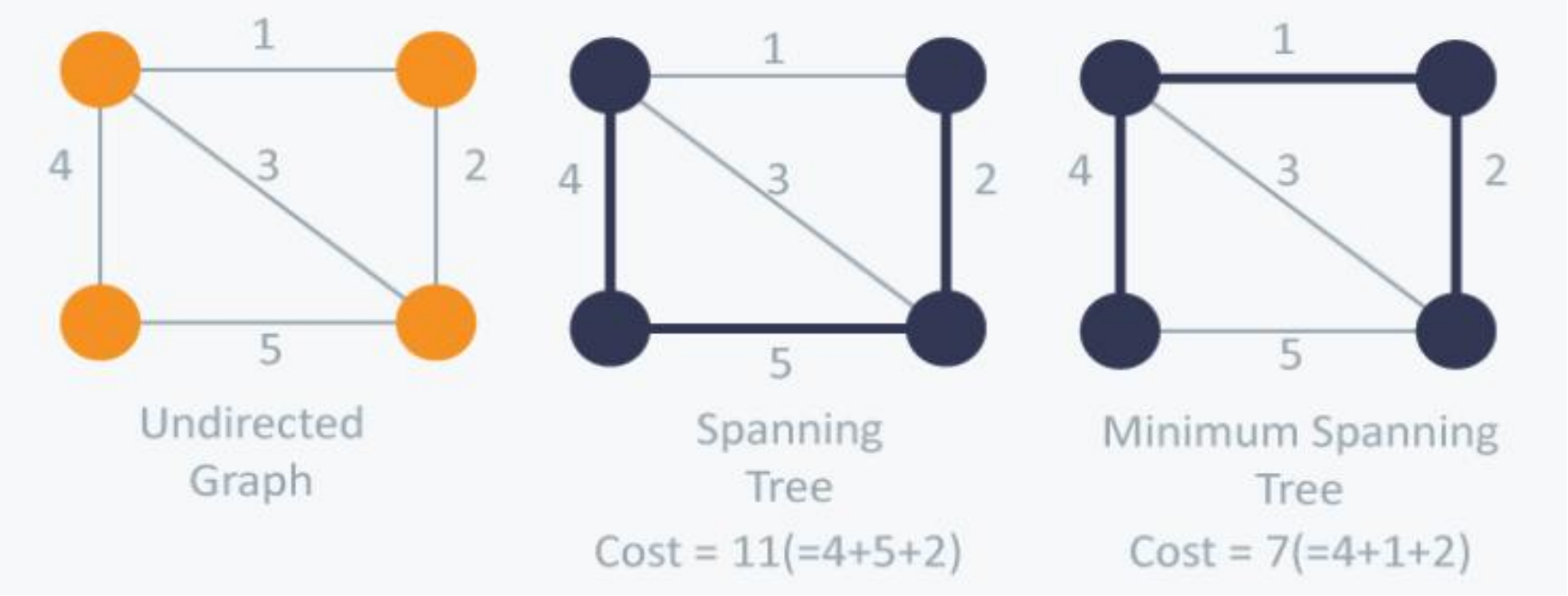
MINIMUM SPANNING TREE

What is a Spanning Tree?

Given an undirected and connected graph G=(V,E), a spanning tree of the graph G is a tree that spans G (that is, it includes every vertex of G) and is a subgraph of G (every edge in the tree belongs to G)

What is a Minimum Spanning Tree?

- The cost of the spanning tree is the sum of the weights of all the edges in the tree. There can be many spanning trees. Minimum spanning tree is the spanning tree where the cost is minimum among all the spanning trees. There also can be many minimum spanning trees.
- Minimum spanning tree has direct application in the design of networks. It is used in algorithms approximating the travelling salesman problem, multi-terminal minimum cut problem and minimum-cost weighted perfect matching. Other practical applications are:
    1. Cluster Analysis
    2. Handwriting recognition
    3. Image segmentation



Types of minimum spanning tree

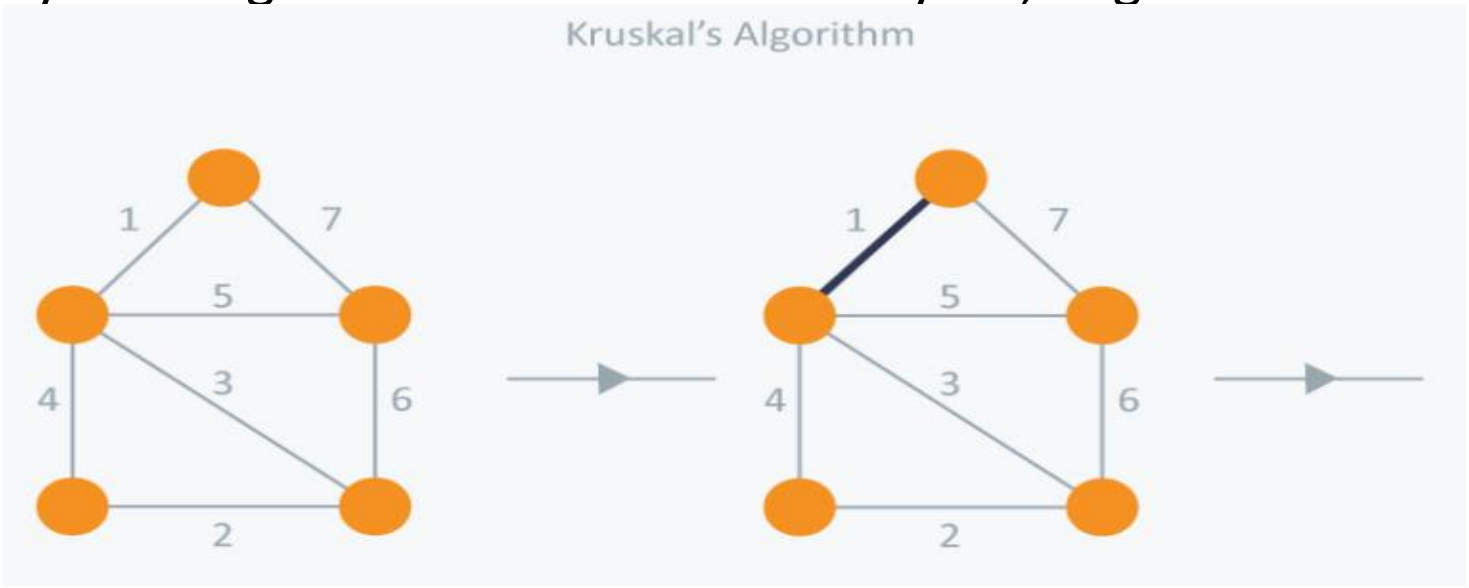
- Kruskal's Algorithm
- Prim's Algorithm

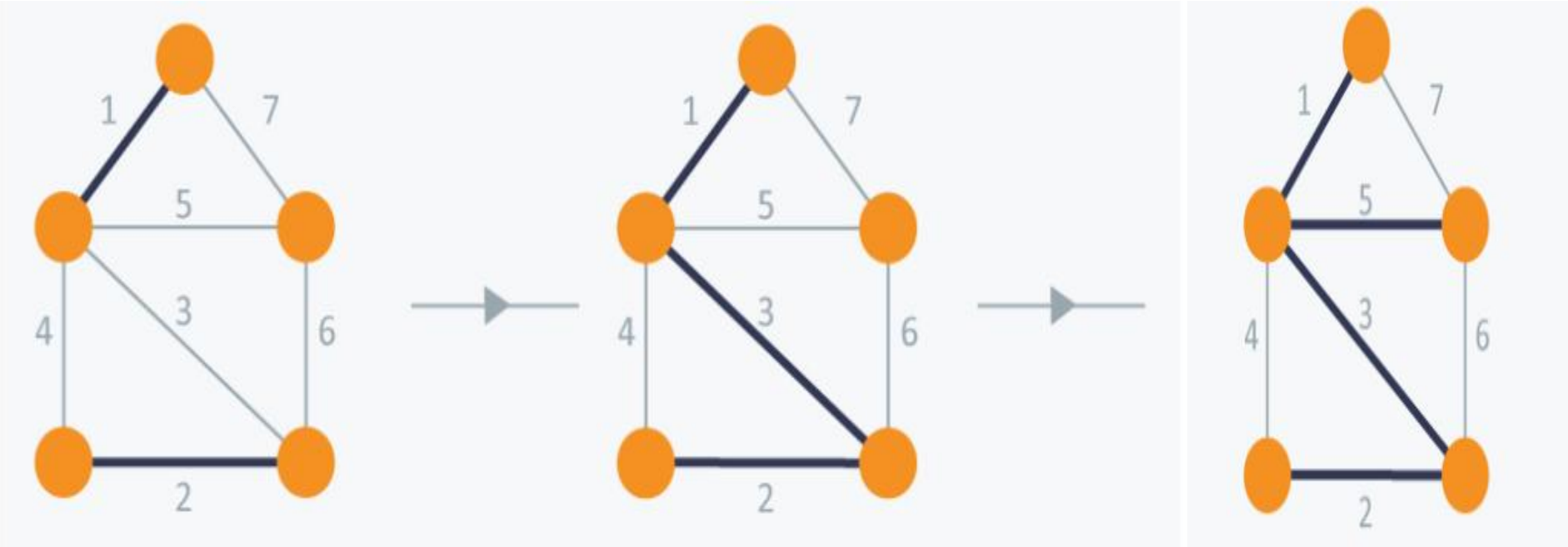
Kruskal’s Algorithm

Kruskal’s Algorithm builds the spanning tree by adding edges one by one into a growing spanning tree. Kruskal's algorithm follows greedy approach as in each iteration it finds an edge which has least weight and add it to the growing spanning tree.

Algorithm Steps:

- Sort the graph edges with respect to their weights.
- Start adding edges to the MST from the edge with the smallest weight until the edge of the largest weight.
- Only add edges which doesn't form a cycle , edges which connect or





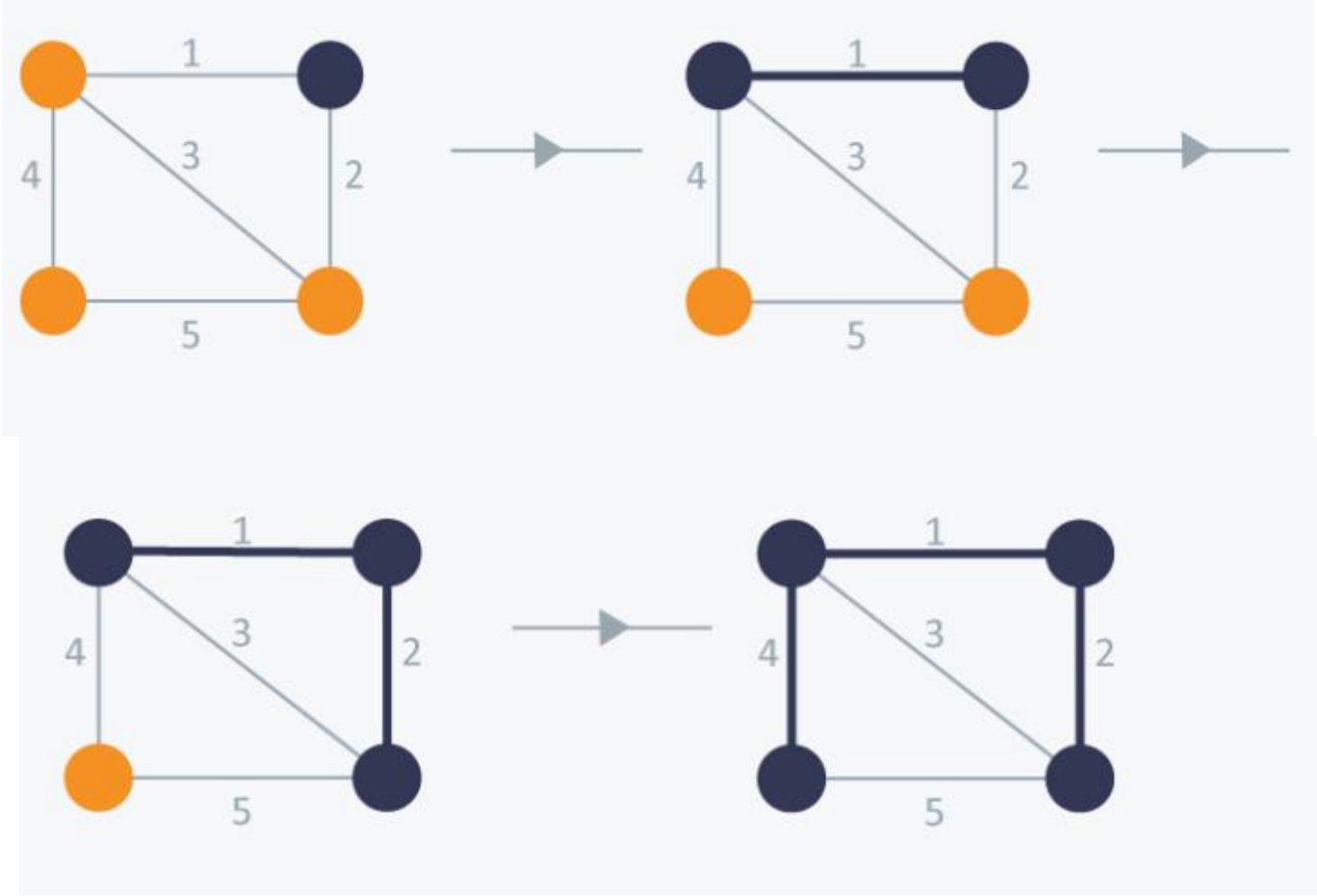
In Kruskal's algorithm, at each iteration we will select the edge with the lowest weight. So, we will start with the lowest weighted edge first i.e., the edges with weight 1. After that we will select the second lowest weighted edge i.e., edge with weight 2. Notice these two edges are totally disjoint. Now, the next edge will be the third lowest weighted edge i.e., edge with weight 3, which connects the two disjoint pieces of the graph. Now, we are not allowed to pick the edge with weight 4, that will create a cycle and we can't have any cycles. So we will select the fifth lowest weighted edge i.e., edge with weight 5. Now the other two edges will create cycles so we will ignore them. In the end, we end up with a minimum spanning tree with total cost 11 (= 1 + 2 + 3 + 5).

Prim's Algorithm

Prim's Algorithm also use Greedy approach to find the minimum spanning tree. In Prim's Algorithm we grow the spanning tree from a starting position. Unlike an **edge** in Kruskal's, we add **vertex** to the growing spanning tree in Prim's.

Algorithm Steps:

- Maintain two disjoint sets of vertices. One containing vertices that are in the growing spanning tree and other that are not in the growing spanning tree.
- Select the cheapest vertex that is connected to the growing spanning tree and is not in the growing spanning tree and add it into the growing spanning tree. This can be done using Priority Queues. Insert the vertices, that are connected to growing spanning tree, into the Priority Queue.
- Check for cycles. To do that, mark the nodes which have been already selected and insert only those nodes in the Priority Queue that are not marked.



In Prim's Algorithm, we will start with an arbitrary node (it doesn't matter which one) and mark it. In each iteration we will mark a new vertex that is adjacent to the one that we have already marked. As a greedy algorithm, Prim's algorithm will select the cheapest edge and mark the vertex. So we will simply choose the edge with weight 1. In the next iteration we have three options, edges with weight 2, 3 and 4. So, we will select the edge with weight 2 and mark the vertex. Now again we have three options, edges with weight 3, 4 and 5. But we can't choose edge with weight 3 as it is creating a cycle. So we will select the

Applications of Graphs

1. Social Networks

- **Graph Representation**: Users are nodes; relationships (friends, followers) are edges.
- **Use Case**:
  - Suggesting friends (common neighbors),
  - Influencer detection (centrality),
  - Community detection.

2. Routing and Navigation

- **Graph Representation**: Locations (cities, intersections) as nodes; roads as edges.
- **Use Case**:
  - GPS systems (Google Maps),
  - Finding shortest or fastest route using algorithms like **Dijkstra**, **A***.

3. Web Page Ranking

- **Graph Representation**: Webpages as nodes, hyperlinks as directed edges.
- **Use Case**:
  - **PageRank algorithm** (used by Google),
  - Finding most "important" pages based on link structure.

4. Network Design

- **Graph Representation**: Devices as nodes; cables, connections as edges.
- **Use Case**:
  - Design minimal-cost networks using **Minimum Spanning Tree (MST)** algorithms (Kruskal's, Prim's),
  - Load balancing and fault tolerance.

5. Dependency Resolution

- **Graph Representation**: Tasks/modules as nodes; dependencies as edges.
- **Use Case**:
  - **Build systems (Makefiles, compilers)** to resolve compilation order,
  - **Package managers** (like npm, pip) resolve and install dependencies using **Topological Sorting**.

Applications of Graph

- **6. Recommendation Systems**
- **Graph Representation**: Users, products as nodes; purchases/ratings as edges.
- **Use Case**:
  - Collaborative filtering using **bipartite graphs**,
  - Finding similar users/items based on graph structure.
- **7. Path Finding & Maze Solving**
- **Graph Representation**: Maze cells or grid tiles as nodes; possible movements as edges.
- **Use Case**:
  - Games, AI, robotics use BFS, DFS, A* for movement and search.
- **8. Cycle Detection**
- **Use Case**:
  - **Deadlock detection** in OS,
  - Detecting circular dependencies in spreadsheets or software packages.
- **9. Knowledge Graphs**
- **Graph Representation**: Entities (people, places, events) as nodes; relationships as edges.
- **Use Case**:
  - Semantic search, AI reasoning, chatbots.
- **10. Modeling State Machines**
- **Graph Representation**: States as nodes; transitions as edges.
- **Use Case**:
  - Finite automata,
  - Software modeling (e.g., user interfaces, game levels).